SkillSwap | Project Report

Web Programming (511) - Milestone 1 - Winter 2026 - German International University

1. Team and Project Idea

Team GYMM	ID	Email
Yahia Mohamed	16004934	yahia.ghitas@student.giu-uni.de
Malak Elsadek	16004523	malak.elsadek@student.giu-uni.de
Maya Ramy	16001727	maya.mohamed@student.giu-uni.de
Ghada Haitham	16005525	ghada.maki@student.giu-uni.de

Repository: https://github.com/malakelsadek/skill-swap.git

Project idea. SkillSwap is an online course marketplace ("Learn a skill. Share a skill."). Instructors publish courses and earn a share of every paid sale; students buy courses one by one or by subscription and can request refunds; the accountant and admin run payroll, the department budget, payouts and refund approvals.

2. Description of the Application

A single-page Angular app. The navigation and pages depend on the role, and route guards protect restricted pages.

- **Visitors:** home, course catalog (search, category filter, sorting), course details, Log in, and Sign up with full validation (email, password of 8+ characters with a digit, picture, card, preferences).
- **Students:** cart, mock checkout (Pay Now / Pay Later), invoices, payment history, refund requests, profile (masked card).
- **Instructors:** add, edit and delete own courses; earnings page with the 70% share of paid sales.
- **Accountant / admin:** department budget with chart, staff payroll and payouts (JSON reports), refund approvals (updates the invoice and budget), and a Users list for the admin.
- **Extras:** AI course chatbot, sound effects, responsive layout, lazy-loaded routes, "Not authorized" page, 158 passing Karma/Jasmine specs.

3. Data Storage: Implementation and Why

There is no backend, so all data lives in the browser. One **StorageService** wraps the three mechanisms from the lecture, and each piece of data goes where its lifetime and sensitivity fit.

Mechanism	What we store	Lifetime / scope	Why, and effect on the user
Local storage	Business data as JSON under sw_ keys (courses, enrollments, invoices, payments, refunds, users, sound, budget).	Until deleted; same browser and origin.	Purchases and refunds persist and appear in the instructor and accountant views; nothing goes over the network.
Session storage	sw_session = {"userId":3} (id only) and the chat history.	While the tab is open; not shared between tabs.	Stays logged in on refresh, logged out when the tab closes (safer on shared computers); no password kept.
Cookie	sw_remember = username only, only if "Remember me" is ticked.	Has an expiry, path "/", SameSite=Lax.	Recognises a returning user with their consent; the tiny value avoids the ~4 KB limit and extra request bytes.

- **Privacy:** session = id only, cookie = username only, never a password; only the last 4 card digits are kept; the cookie needs consent (GDPR) and the chat history disappears with the tab.
- **Security:** passwords are stored as a hash (modified DJB2), compared at login, and a failed login shows one generic message.
- **Time saving:** data persists, login survives a refresh, large data stays out of cookies (not sent with every request), and lazy-loaded routes shrink the first download.
- **Reliability:** corrupted JSON returns null instead of crashing, and demo data is seeded only when storage is empty.

Limitations. DJB2 is unsalted and not cryptographic, storage is readable from DevTools (Figures 1-3) and limited to one browser, and the cookie cannot be HttpOnly. A real version needs a server, bcrypt/Argon2 and secure cookies.

Sample run. A student logs in with Remember me, buys courses and requests a refund; the accountant approves it, the invoice becomes refunded and the budget updates. All roles read the same records.

4. Screenshots of Sample Runs

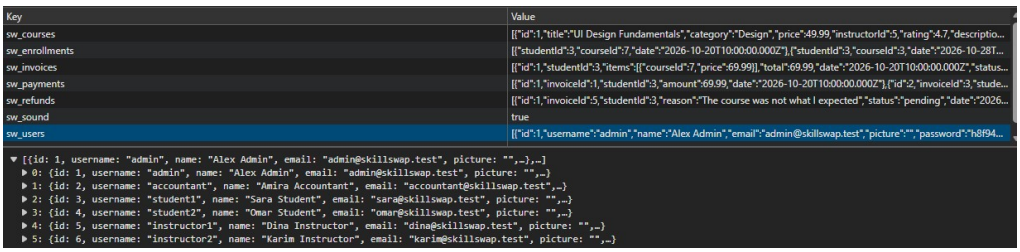


Fig. 1 Local storage: sw_ keys for courses, enrollments, invoices, payments, refunds, users, sound.

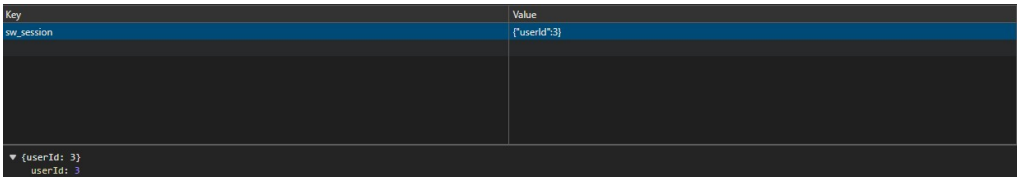


Fig. 2 Session storage: sw_session holds only {"userId":3}.

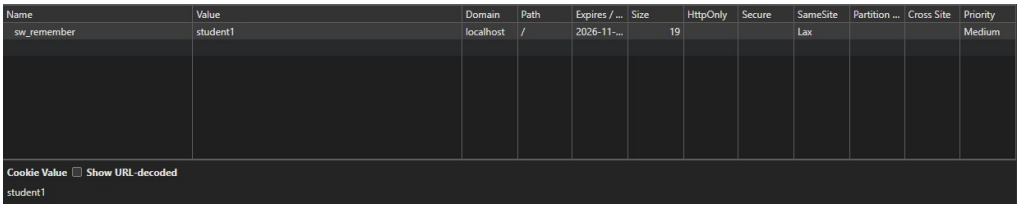


Fig. 3 Cookie: sw_remember = student1 with expiry, path / and SameSite=Lax.

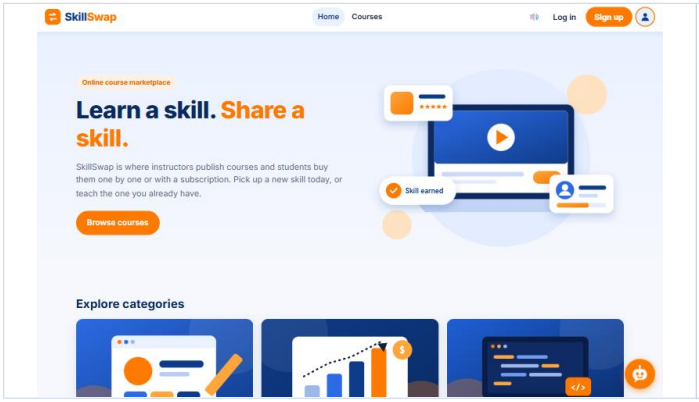


Fig. 4 Home page

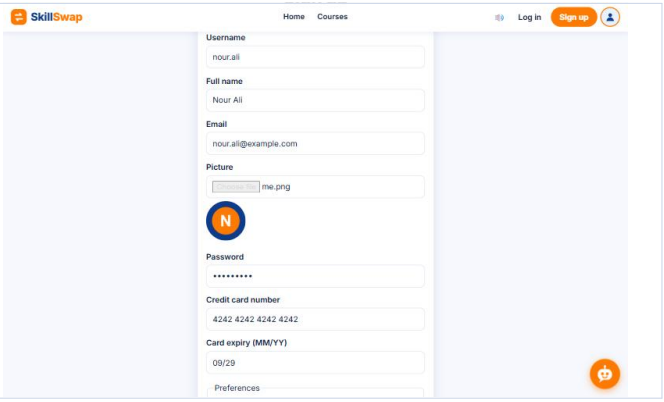


Fig. 5 Sign up form

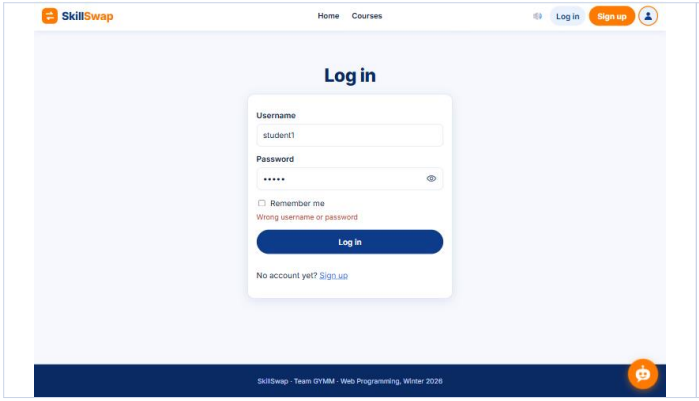


Fig. 6 Generic login error

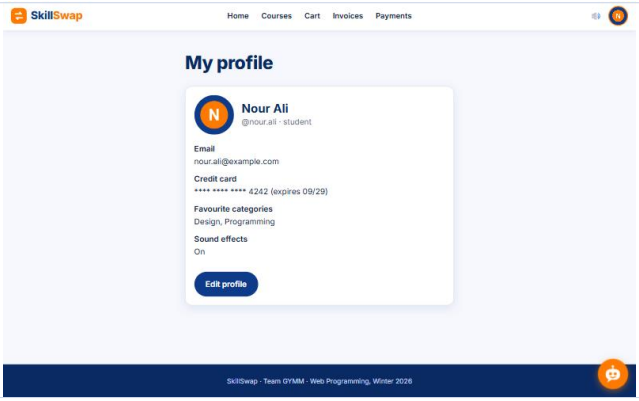


Fig. 7 Profile with masked card

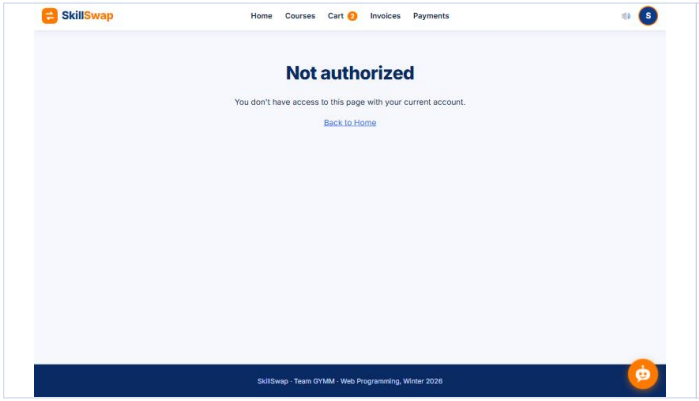


Fig. 8 Role guard: Not authorized

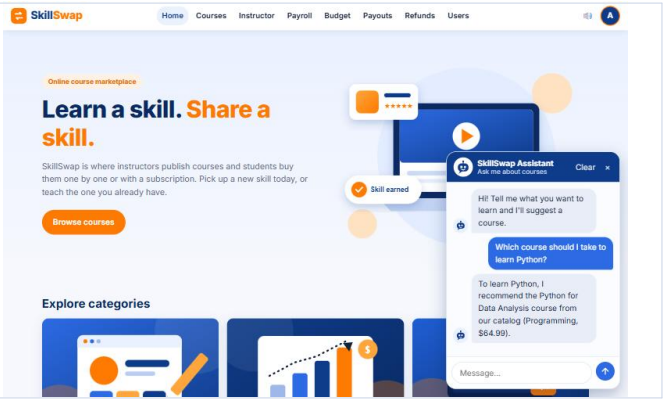


Fig. 9 Course chatbot (bonus)

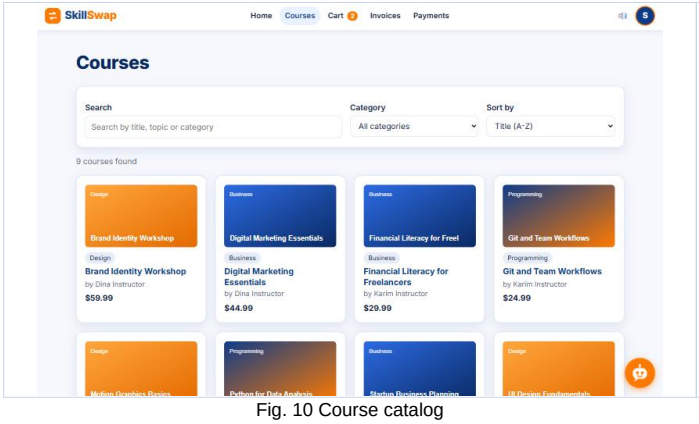


Fig. 10 Course catalog

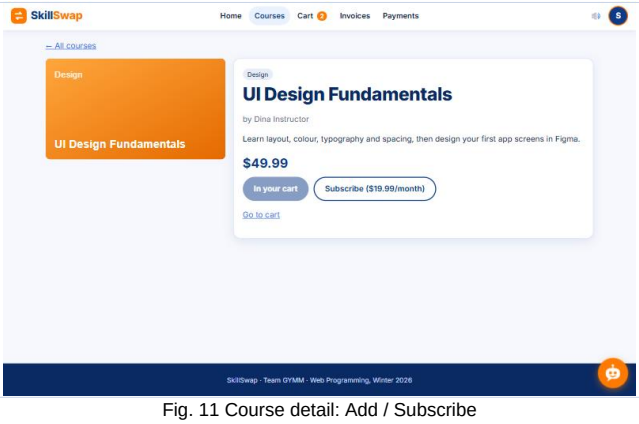


Fig. 11 Course detail: Add / Subscribe

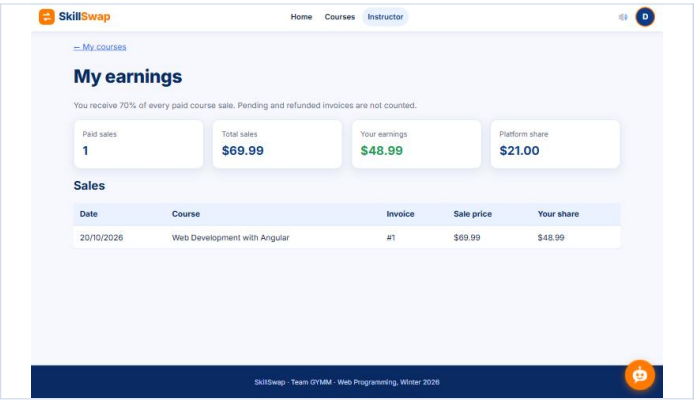


Fig. 12 Instructor earnings (70% share)

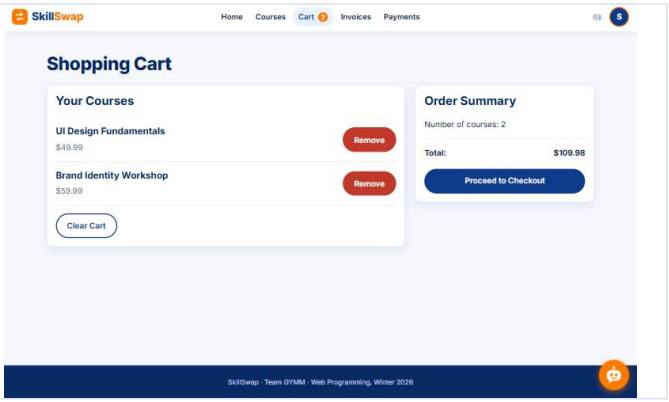


Fig. 13 Cart and order summary

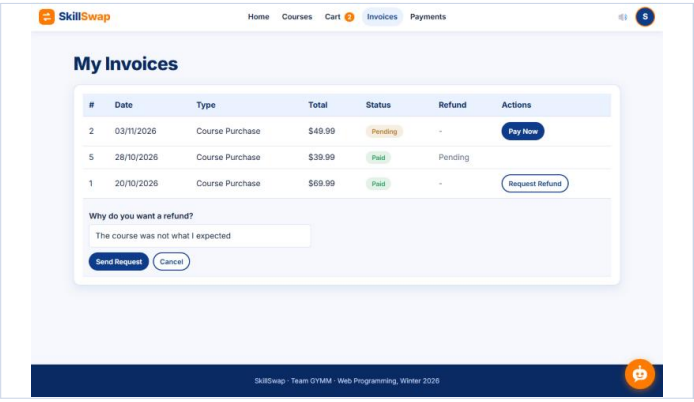


Fig. 14 Refund request

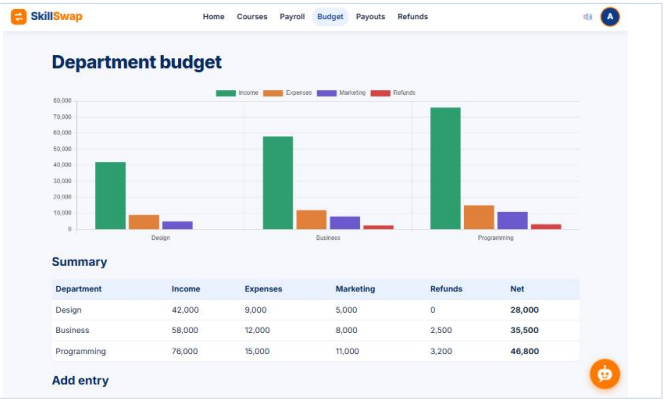


Fig. 15 Department budget chart

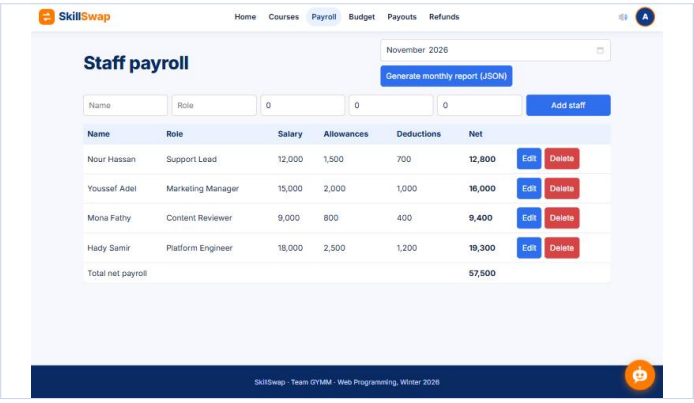


Fig. 16 Staff payroll

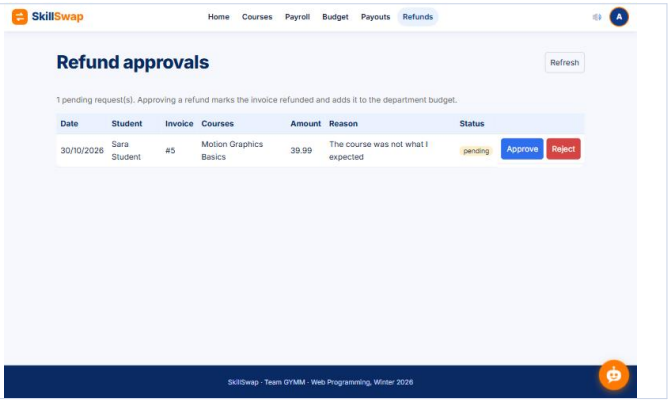


Fig. 17 Refund approvals

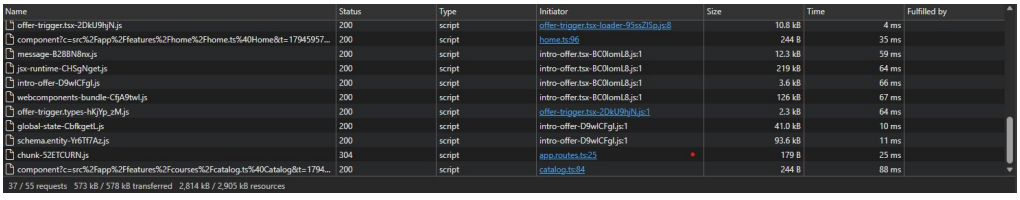


Fig. 18 Lazy loading: the catalog code chunk is requested only when the Courses page is opened.



Fig. 19 Karma/Jasmine: 158 specs, 0 failures.